

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA AN TOÀN THÔNG TIN

BÁO CÁO CHUYÊN ĐỀ

An toàn Thông tin

Đề tài

**Nghiên cứu cơ chế phát hiện tấn công leo
thang
đặc quyền trong môi trường Multi-tenant
Kubernetes
dựa trên phân tích System Calls**

Giảng viên hướng dẫn : TS. Vũ Thị Vân

Học viên thực hiện : Nguyễn Ngọc Hoài — CHAT4P006

Lớp : CHAT4P

Hà Nội, tháng 6 năm 2026

Tóm tắt

Trong môi trường *multi-tenant Kubernetes*, nhiều người thuê (tenant) bị cô lập bằng không gian tên (namespace) nhưng vẫn *chia sẻ chung một nhân* hệ điều hành; do đó lời gọi hệ thống (syscall) trở thành bề mặt tấn công trọng yếu cho *leo thang đặc quyền* và thoát container—kẻ tấn công vượt qua ranh giới tenant để chiếm quyền trên máy chủ. Báo cáo chuyên đề này nghiên cứu một *cơ chế phát hiện* các tấn công leo thang đặc quyền dựa trên phân tích syscall thời gian thực. Cơ chế kế thừa thiết kế lỗi phát hiện bất thường của công trình tham chiếu DeSFAM [1] (SyscallAD: Bộ mã hóa biến phân tự động VAE hợp nhất với Rừng cô lập iForest), trong đó các đặc trưng *hiện diện của syscall đặc quyền* (`ptrace`, `setuid/capset`, `setns`, `unshare`, `mount`, `bpf`, nạp mô-đun nhân) chính là tín hiệu hành vi của leo thang đặc quyền. Mô hình được huấn luyện trên bộ dữ liệu công khai DongTing [2] và được hiện thực hóa thành một bộ phát hiện chạy trực tiếp: cảm biến syscall dùng Tetragon [3] (eBPF) giám sát theo namespace trên cụm Kubernetes, đẩy cảnh báo qua Prometheus/Grafana. Toàn bộ quy trình huấn luyện và suy luận được đóng gói bằng Docker để bảo đảm khả năng tái lập. Trên DongTing, mô hình VAE đạt AUC kiểm định 0,944; tập hợp VAE + iForest đạt AUC kiểm định $\approx 0,95$ và AUC trên tập kiểm tra 0,835 (F1 0,417 tại ngưỡng bảo thủ chọn trên tập kiểm định). Khi triển khai trực tiếp, điểm bất thường tách biệt rõ giữa khối lượng công việc lành tính ($\leq 0,19$) và các kịch bản leo thang đặc quyền/thoát container mô phỏng theo các họ CVE thực (CVE-2021-4034, CVE-2022-0847, CVE-2019-5736) trong namespace thử nghiệm (0,278–0,930), cho phép một ngưỡng vận hành $T_0 = 0,25$ phát hiện tấn công thời gian thực. Báo cáo cũng đối chiếu trung thực cơ chế đề xuất với số liệu công bố của DeSFAM, chỉ ra các điểm tương đồng và khác biệt, cùng bài học then chốt về căn chỉnh không gian đặc trưng giữa huấn luyện và phục vụ.

Từ khóa: leo thang đặc quyền, multi-tenant Kubernetes, bảo mật container, lời gọi hệ thống, eBPF, Tetragon, phát hiện bất thường, Bộ mã hóa biến phân tự động (VAE), Rừng cô lập, DongTing, DeSFAM.

Mục lục

1	Mở đầu	1
1.1	Bối cảnh và động cơ	1
1.2	Mục tiêu và phạm vi báo cáo	1
1.3	Nội dung và bố cục báo cáo	2
2	Cơ sở lý thuyết và công trình liên quan	2
2.1	Leo thang đặc quyền trong multi-tenant Kubernetes	2
2.2	Bảo mật lời gọi hệ thống cho container	3
2.3	Sinh danh sách syscall và lọc động	3
2.4	Phát hiện xâm nhập dựa trên chuỗi syscall	3
2.5	Bộ dữ liệu DongTing	3
2.6	eBPF và Tetragon	3
2.7	Framework tham chiếu DeSFAM và ba mô-đun	4
2.8	Vị trí của chuyên đề	4
3	Thiết kế cơ chế phát hiện	4
3.1	Tổng quan cách tiếp cận	4
3.2	Kiến trúc tổng thể	5
3.3	Dữ liệu và tiền xử lý	5
3.4	Trích đặc trưng theo cửa sổ trượt	6
3.5	Mô hình SyscallAD	6
3.6	Chọn ngưỡng (không rò rỉ)	7
3.7	Mở rộng: bộ phát hiện trực tiếp	7
4	Triển khai và thiết lập thực nghiệm	7
4.1	Nguyên tắc: mọi công cụ chạy bằng Docker	7
4.2	Huấn luyện ngoại tuyến	8
4.3	Cấu hình tái lập (ghi trong <code>results.json</code>)	8
4.4	Triển khai trực tiếp trên multi-tenant Kubernetes	8
4.5	Khối lượng công việc và kịch bản tấn công	9
4.6	Quan sát	9
5	Kết quả và đánh giá	9
5.1	Hiệu lực phát hiện ngoại tuyến trên DongTing	9
5.1.1	Hiệu năng từng mô hình và tập hợp	9
5.1.2	Đối chiếu với công trình tham chiếu	9

5.2	Triển khai trực tiếp trên multi-tenant Kubernetes	10
5.3	Điều kiện thành công của cơ chế	11
5.4	Thảo luận và đánh giá	11
5.4.1	Cơ chế đạt được gì so với công trình tham chiếu	11
5.4.2	Ý nghĩa của khoảng cách ngưỡng	11
5.4.3	Lựa chọn thiết kế khác với công trình tham chiếu	12
5.4.4	Hạn chế và đe dọa tính hợp lệ	12
5.4.5	Hướng cải thiện	12
6	Kết luận	12
A	Phụ lục Tái lập Kết quả	15
A.1	Môi trường	15
A.2	Huấn luyện ngoại tuyến (tái lập Bảng 2)	16
A.3	Triển khai trực tiếp (tái lập Bảng 4)	16
A.4	Kiểm tra tính nhất quán đặc trưng	16
A.5	Biên dịch báo cáo này (Docker)	16

Danh mục hình vẽ

1	Kiến trúc cơ chế phát hiện (kế thừa thiết kế tham chiếu DeSFAM): cảm biến syscall (eBPF/Tetragon) giám sát theo namespace → trích đặc trưng theo cửa sổ trượt → SyscallAD (VAE + iForest) → chấm điểm rủi ro và phản ứng. Mũi tên ngoại tuyến: huấn luyện trên DongTing; mũi tên trực tuyến: phục vụ trên multi-tenant Kubernetes.	5
2	Quy trình kỹ thuật đặc trưng: từ chuỗi syscall thô → lọc theo bảng chữ cái cảm biến → cửa sổ trượt (15, 3) → véc-tơ 43 chiều.	6
3	Ánh xạ các syscall đặc quyền (đặc trưng <code>disc</code>) sang nhóm hành vi leo thang đặc quyền và ví dụ CVE thực. Mỗi nhóm là một vector tấn công vượt ranh giới tenant; bit hiện diện của một syscall đặc quyền là tín hiệu phát hiện cốt lõi của cơ chế.	7
4	Phân tách điểm bất thường khi phục vụ trực tiếp qua Tetragon trên Kubernetes. Khối lượng lạnh tính (MongoDB) nằm dưới ngưỡng vận hành $T_0 = 0,25$, các kịch bản tấn công nằm trên—khoảng trống đủ rộng để một ngưỡng duy nhất tách sạch hai lớp.	11

Danh mục bảng

1	Cấu hình thí nghiệm tái lập (trích <code>results.json</code>).	8
2	Hiệu năng SyscallAD trên tập kiểm tra DongTing (cửa sổ 15/3, 43 chiều). Ngưỡng chọn trên tập kiểm định.	9
3	Cơ chế đề xuất so với số liệu công bố của công trình tham chiếu DeSFAM (lỗi phát hiện bất thường).	10
4	Phân tách điểm bất thường khi phục vụ trực tiếp (Tetragon/K8s). . . .	10

1 Mở đầu

1.1 Bối cảnh và động cơ

Kubernetes đã trở thành nền tảng điều phối container phổ biến nhất cho điện toán đám mây hiện đại. Trong mô hình triển khai *multi-tenant* (nhiều người thuê), các tenant được cô lập *mức logic* bằng không gian tên (namespace), hạn ngạch tài nguyên và chính sách mạng—nhưng tất cả vẫn *chia sẻ chung một nhân* hệ điều hành của nút (node). Khác với máy ảo, ranh giới cô lập giữa các tenant vì thế mỏng hơn nhiều và hội tụ tại một điểm duy nhất: giao diện lời gọi hệ thống (syscall)—ranh giới giữa không gian người dùng và nhân [4, 5].

Hệ quả là *leo thang đặc quyền* (privilege escalation) và *thoát container* (container escape) trở thành lớp mối đe dọa nguy hiểm nhất trong môi trường multi-tenant: một tenant bị xâm phạm có thể lạm dụng syscall đặc quyền để chiếm quyền root trên nút và phá vỡ cô lập, ảnh hưởng tới mọi tenant khác trên cùng nút [6]. Các họ lỗ hổng tiêu biểu—PwnKit (CVE-2021-4034), Dirty Pipe (CVE-2022-0847), thoát `runc` (CVE-2019-5736)—đều để lại *dấu vết hành vi* ở tầng syscall (gọi `ptrace`, `setuid/capset`, `unshare/setns`, `mount`, nạp mô-đun nhân, `bpf`...). Các cơ chế tĩnh như hồ sơ seccomp khó duy trì, dễ chặn nhầm thao tác hợp lệ hoặc bỏ sót tấn công nhiều bước, và không nhận thức được *chuỗi* hành vi đặc trưng cho leo thang đặc quyền.

Một hướng tiếp cận thích nghi đã được công trình tham chiếu DeSFAM [1] đề xuất: kết hợp lập hồ sơ syscall lai (tĩnh + động), phát hiện bất thường bằng học máy độ trễ thấp (mô-đun *SyscallAD* dùng VAE và iForest), tính điểm rủi ro theo ngữ cảnh (ánh xạ MITRE ATT&CK, tương quan CVE) và thực thi trong nhân qua eBPF/LSM. Tuy nhiên DeSFAM đánh giá trên hạ tầng riêng, không công bố mã nguồn đầy đủ và không đặt trọng tâm vào ngữ cảnh *multi-tenant Kubernetes*. Chuyên đề này kế thừa thiết kế lõi phát hiện của DeSFAM làm *tham chiếu* và xây dựng một cơ chế phát hiện leo thang đặc quyền vận hành trực tiếp trong Kubernetes.

1.2 Mục tiêu và phạm vi báo cáo

Báo cáo chuyên đề này trình bày một *cơ chế phát hiện tấn công leo thang đặc quyền* cho môi trường multi-tenant Kubernetes dựa trên phân tích syscall, xây dựng trên nền các nghiên cứu liên quan—trọng tâm là công trình tham chiếu DeSFAM [1]. Đây là một báo cáo môn học theo hướng nghiên cứu chủ đề: hệ thống hóa cơ sở lý thuyết, thiết kế và hiện thực hóa cơ chế, rồi đánh giá thực nghiệm.

Trọng tâm: mô-đun SyscallAD. Framework tham chiếu DeSFAM gồm *ba* mô-đun (giai đoạn) tích hợp: (1) *Lập danh sách truy cập syscall lai* và sinh hồ sơ seccomp; (2) *SyscallAD*—giám sát thời gian thực và phát hiện bất thường syscall bằng tập hợp VAE + iForest; (3) *Thực thi thích ứng* (chấm điểm rủi ro, ánh xạ MITRE ATT&CK, chặn trong nhân bằng eBPF/LSM). Báo cáo này **đặt trọng tâm vào mô-đun SyscallAD** (Giai đoạn 2)—lõi phát hiện leo thang đặc quyền dựa trên hành vi syscall; hai mô-đun còn lại (Giai đoạn 1 và Giai đoạn 3) chỉ được *điểm qua* để giữ tính trọn vẹn của bức tranh kiến trúc, chứ không phải đối tượng nghiên cứu chính. Cụ thể, báo cáo:

- (i) Xây dựng lõi phát hiện bất thường syscall (VAE + iForest, hợp nhất tập hợp) theo thiết kế tham chiếu SyscallAD của DeSFAM [1], trong đó các đặc trưng *hiện diện của syscall đặc quyền* đóng vai trò tín hiệu chính cho leo thang đặc quyền; huấn luyện và kiểm chứng ngoại tuyến trên bộ dữ liệu công khai DongTing [2];
- (ii) Hiện thực hóa cơ chế thành một bộ phát hiện *trực tiếp* trong Kubernetes: thu thập syscall theo namespace (tenant) bằng Tetragon [3] (eBPF), trích đặc trưng theo cửa sổ trượt giống hệt lúc huấn luyện, chấm điểm và phát cảnh báo qua Prometheus/Grafana;
- (iii) Đánh giá khả năng phân tách lành tính/tấn công trên các kịch bản leo thang đặc quyền mô phỏng theo CVE thực, và đối chiếu trung thực với số liệu công bố của công trình tham chiếu.

1.3 Nội dung và bố cục báo cáo

Báo cáo tập trung vào lớp phát hiện (không bao gồm khâu chặn nội tuyến trong nhân) và trình bày các nội dung chính sau:

- Một cơ chế phát hiện leo thang đặc quyền cho multi-tenant Kubernetes dựa trên phân tích syscall, với quy trình huấn luyện *hoàn toàn bằng Docker*, có thể chạy lại từ đầu và lưu vết kết quả số (`results.json`).
- Một bộ phát hiện trực tiếp dựa trên Tetragon/eBPF giám sát theo namespace trên Kubernetes, cho thấy thiết kế tham chiếu DeSFAM khả thi trong môi trường vận hành thực.
- Một đối chiếu *trung thực* giữa cơ chế đề xuất và số liệu công bố của DeSFAM, cùng bài học về căn chỉnh “bảng chữ cái cảm biến” (sensor alphabet) giữa huấn luyện và phục vụ.

Phần còn lại của báo cáo được tổ chức như sau: Mục 2 trình bày cơ sở lý thuyết và công trình liên quan; Mục 3 mô tả thiết kế cơ chế phát hiện; Mục 4 trình bày triển khai và thiết lập thực nghiệm bằng Docker/Kubernetes; Mục 5 báo cáo kết quả và đánh giá (bao gồm thảo luận và hạn chế); Mục 6 kết luận.

2 Cơ sở lý thuyết và công trình liên quan

2.1 Leo thang đặc quyền trong multi-tenant Kubernetes

Trong cụm Kubernetes nhiều người thuê, các tenant được cô lập mức logic bằng namespace nhưng chia sẻ chung nhân của nút. Bề mặt nguy hiểm nhất là *leo thang đặc quyền* và *thoát container*: tận dụng lỗ hổng nhân hoặc cấu hình sai (capabilities dư thừa, mount host-path, ptrace, unshare/ setns, nạp mô-đun nhân) để vượt ranh giới tenant và chiếm quyền trên nút [6]. Mọi con đường này đều biểu hiện thành các *chuỗi syscall đặc quyền* đặc trưng, nên phân tích syscall là điểm quan sát tự nhiên để phát hiện chúng.

2.2 Bảo mật lời gọi hệ thống cho container

Trong mô hình chia sẻ nhân, nhân Linux phơi bày hàng trăm syscall, phần lớn không cần thiết cho một khối lượng công việc cụ thể, làm phình to bề mặt tấn công [7]. Các cơ chế truyền thống (seccomp, AppArmor, SELinux) lọc theo chính sách tĩnh nên: (1) không thích nghi với hành vi thời gian chạy; (2) thiếu nhận thức về *chuỗi* syscall nên mù với tấn công nhiều bước; và (3) tốn chi phí cấu hình thủ công ở quy mô lớn.

2.3 Sinh danh sách syscall và lọc động

Confine [8] dùng phân tích tĩnh để rút gọn hồ sơ seccomp ($SRR \approx 47,6\%$) nhưng không thích nghi thời gian chạy. Prof-gen [9] kết hợp theo dõi lúc khởi động với đồ thị lời gọi tĩnh, vẫn bỏ sót thay đổi hành vi về sau. Optimus [10] và KubeRosy [11] dùng eBPF để cập nhật chính sách thời gian chạy, song Optimus cần phê duyệt thủ công và KubeRosy gắn chặt với Kubernetes. NIMOS [12] đánh giá giá trị của lọc dựa trên chuỗi. DeSFAM định vị mình ở chỗ hợp nhất cả ba: lập hồ sơ lai, phát hiện bất thường học máy, và thực thi trong nhân.

2.4 Phát hiện xâm nhập dựa trên chuỗi syscall

Hướng phát hiện bất thường dựa trên chuỗi syscall có lịch sử lâu dài, khởi nguồn từ “a sense of self” [13, 14] và các mô hình thay thế [15], cùng cảnh báo kinh điển về tấn công bất chước [16]. Các hướng hiện đại gồm phương pháp ngữ nghĩa [17], túi lời gọi (BoSC) cho container [18], và học sâu một lớp/biến phân như DAGMM [19], Deep SVDD [20], HIDS học sâu [21]. Riêng cho container có các đánh giá phát hiện bất thường [22] và ngữ cảnh hóa syscall [23], cũng như tập hợp Random/Isolation Forest [24]. SyscallAD của DeSFAM nằm trong dòng này: hợp nhất VAE [25] (lỗi tái cấu trúc) với iForest [26] (ngoại lệ thống kê), bổ sung khớp mẫu lạnh tính bằng PrefixSpan [27].

2.5 Bộ dữ liệu DongTing

DongTing [2] là bộ dữ liệu quy mô lớn gồm các chuỗi syscall của nhân Linux được gán nhãn bình thường/độc hại (theo các bản vá lỗi syzkaller), thích hợp cho huấn luyện và xác nhận bộ phát hiện bất thường. Chuyên đề này dùng DongTing làm nguồn dữ liệu ngoại tuyến duy nhất, đúng như DeSFAM.

2.6 eBPF và Tetragon

eBPF cho phép nạp logic giám sát/thực thi an toàn vào nhân tại các điểm vào syscall với chi phí thấp [28]. DeSFAM dùng eBPF/LSM để chặn nội tuyến. Trong chuyên đề này, thay vì tự viết chương trình eBPF, chúng tôi dùng Tetragon [3] (thuộc hệ sinh thái Cilium [29])—một lớp quan sát và thực thi bảo mật dựa trên eBPF cho Kubernetes—làm *cảm biến syscall* phát sự kiện qua gRPC. Đây là lựa chọn thực dụng để hiện thực hóa lớp thu thập của DeSFAM mà không cần sửa nhân.

2.7 Framework tham chiếu DeSFAM và ba mô-đun

DeSFAM (*Dynamic eBPF-Driven Syscall Filtering and Anomaly Mitigation*) [1] là một framework bảo mật thời gian thực, nhẹ cho container, gồm *ba mô-đun* (giai đoạn) tích hợp:

- Mô-đun 1. Lập danh sách truy cập syscall lai và sinh hồ sơ seccomp.** Kết hợp phân tích tĩnh (đồ thị lời gọi từ binary ELF) với lập hồ sơ động (eBPF dưới khối lượng công việc chuẩn) và lọc theo thông tin CVE để sinh hồ sơ seccomp tối thiểu. *Chỉ điểm qua trong báo cáo này.*
- Mô-đun 2. SyscallAD—giám sát thời gian thực và phát hiện bất thường.** Phân tích chuỗi syscall theo cửa sổ trượt bằng tập hợp VAE + Isolation Forest (kèm khai phá mẫu lãnh tính PrefixSpan và chọn ngưỡng) để phát hiện leo thang đặc quyền, thoát container và chèn syscall—kể cả khi lạm dụng các syscall được phép. **Đây là mô-đun trọng tâm của chuyên đề.**
- Mô-đun 3. Thực thi thích ứng và quản lý chính sách.** Chấm điểm rủi ro (ánh xạ MITRE ATT&CK, mức nghiêm trọng tích lũy, khớp mẫu khai thác) rồi chặn/giới hạn tốc độ/ghi nhật ký trong nhân bằng eBPF gắn vào LSM hook. *Chỉ điểm qua trong báo cáo này.*

Chuyên đề nghiên cứu sâu **Mô-đun 2 (SyscallAD)**: hệ thống hóa thiết kế, hiện thực hóa lỗi phát hiện và đánh giá thực nghiệm; hai mô-đun còn lại được trình bày vắn tắt ở trên chỉ nhằm định vị SyscallAD trong tổng thể framework.

2.8 Vị trí của chuyên đề

Chuyên đề kế thừa DeSFAM như một *công trình tham chiếu* cho thiết kế lỗi phát hiện bất thường syscall, và vận dụng nó để xây dựng một *cơ chế phát hiện leo thang đặc quyền* cho ngữ cảnh multi-tenant Kubernetes—một ngữ cảnh mà DeSFAM không đặt trọng tâm. Đóng góp nằm ở chỗ: (i) định vị các đặc trưng syscall đặc quyền làm tín hiệu cho leo thang đặc quyền; (ii) hiện thực hóa cơ chế thành một bộ phát hiện trực tiếp giám sát theo namespace trên Tetragon/Kubernetes, lấp khoảng trống “từ mô phỏng đến triển khai”; và (iii) đối chiếu trung thực hiệu năng với số liệu công bố của công trình tham chiếu.

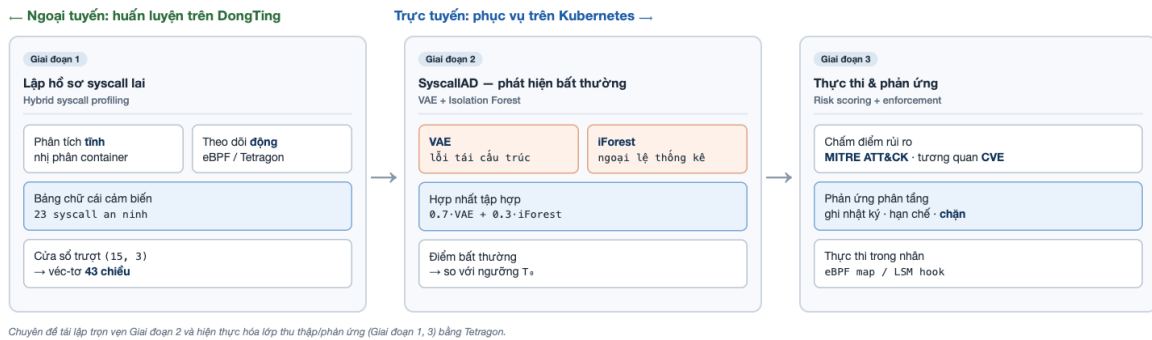
3 Thiết kế cơ chế phát hiện

3.1 Tổng quan cách tiếp cận

Cơ chế phát hiện leo thang đặc quyền được thiết kế theo nguyên tắc *huấn luyện-rời-phục-vụ*: lỗi phát hiện bất thường syscall (SyscallAD) kế thừa thiết kế của công trình tham chiếu DeSFAM [1], với bộ dữ liệu, định nghĩa đặc trưng và giao thức đánh giá được cố định để có thể đối chiếu trực tiếp với số liệu công bố. Thành phần phục vụ trực tiếp giám sát theo namespace (tenant) trên Kubernetes nhằm kiểm chứng tính khả thi triển khai của cơ chế.

3.2 Kiến trúc tổng thể

Hình 1 mô tả kiến trúc ba mô-đun (giai đoạn) theo công trình tham chiếu: (1) lập hồ sơ/danh sách truy cập syscall lại; (2) phát hiện bất thường *SyscallAD*; (3) thực thi thích ứng/phản ứng. **Trọng tâm của chuyên đề là mô-đun SyscallAD (Giai đoạn 2)**; đây cũng là phần được hiện thực hóa và đánh giá trọn vẹn. Hai mô-đun còn lại chỉ được *điểm qua*: Giai đoạn 1 (sinh danh sách truy cập tĩnh+động, hồ sơ seccomp, lọc theo CVE) được tóm tắt ở Mục 2 và không được hiện thực hóa lại; ở Giai đoạn 3 chuyên đề chỉ tận dụng lớp thu thập/cảnh báo (Tetragon ghi nhật ký, phát cảnh báo), còn cơ chế chấm điểm rủi ro (ánh xạ MITRE ATT&CK) và chặn nội tuyến trong nhân bằng eBPF/LSM được nêu là hướng mở rộng chứ không thuộc phạm vi nghiên cứu chính.



Hình 1: Kiến trúc cơ chế phát hiện (kế thừa thiết kế tham chiếu DeSFAM): cảm biến syscall (eBPF/Tetragon) giám sát theo namespace → trích đặc trưng theo cửa sổ trượt → SyscallAD (VAE + iForest) → chấm điểm rủi ro và phản ứng. Mũi tên ngoại tuyến: huấn luyện trên DongTing; mũi tên trực tuyến: phục vụ trên multi-tenant Kubernetes.

3.3 Dữ liệu và tiền xử lý

Nguồn dữ liệu ngoại tuyến là bản phát hành DongTing [2] (18,9 nghìn chuỗi được gán nhãn, các tệp .log kèm bảng phân chia Baseline.xlsx). Chuỗi bình thường lưu dưới dạng *ID syscall* phân tách bằng dấu gạch đứng; chuỗi tấn công lưu dưới dạng *tên syscall*. Chúng tôi dùng bảng syscall_64.tbl (x86-64, bỏ các dòng ABI x32) để ánh xạ tên ↔ ID nhất quán. Phân chia theo bộ dữ liệu: DTDS-train / DTDS-validation / DTDS-test.

Bảng chữ cái cảm biến (sensor alphabet). Đây là quyết định thiết kế *then chốt* để tái lập thành công khâu triển khai trực tiếp. Chính sách truy vết trực tiếp của Tetragon chỉ phát ra **23 syscall liên quan an ninh** (execve, clone, setuid/setgid/capset, openat, unlinkat, renameat2, mount, unshare, setns, pivot_root, socket, connect, bind, mprotect, mmap, prctl, memfd_create, splice, ptrace, init/finit_module, bpf—không gồm read/write/close/futex). Do đó khâu huấn luyện cũng lọc DongTing về đúng tập này *trước* khi cắt cửa sổ, để huấn luyện và phục vụ chia sẻ một không gian đặc trưng. Nếu huấn luyện trên toàn bộ vũ trụ syscall, mọi cửa sổ trực tiếp sẽ suy biến (lành tính $\approx$ tấn công).

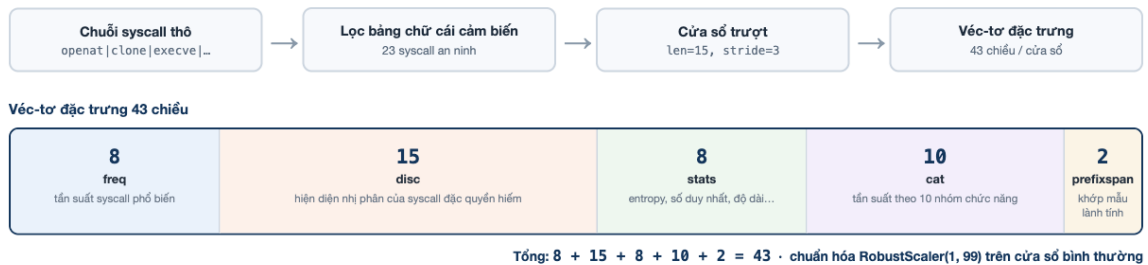
3.4 Trích đặc trưng theo cửa sổ trượt

Mỗi cửa sổ trượt (độ dài 15, bước nhảy 3—giá trị mặc định) được biểu diễn thành véc-tơ **43 chiều** (Hình 2):

$$[\text{freq}_8 \mid \text{disc}_{15} \mid \text{stats}_8 \mid \text{cat}_{10} \mid \text{prefixspan}_2].$$

- **freq (8)**: tần suất trong cửa sổ của các syscall phổ biến.
- **disc (15)**: *hiện diện nhị phân* của các syscall đặc quyền hiếm (ptrace/splice/unshare/setns/mount, đây là *tín hiệu cốt lõi cho leo thang đặc quyền* (Hình 3), và bền với pha loãng: một ptrace đơn lẻ trong cửa sổ vẫn bật bit dù bị lẫn giữa nhiều syscall thông thường.
- **stats (8)**: entropy, số phần tử duy nhất, độ dài, tần suất lớn nhất...
- **cat (10)**: tần suất chuẩn hóa theo nhóm chức năng (tiến trình, tệp, bộ nhớ, mạng, tín hiệu, thời gian, IPC, an ninh, sự kiện I/O, khác).
- **prefixspan (2)**: [cờ-khớp, độ-dài-khớp/L] từ việc đối sánh cửa sổ với một *Cơ sở dữ liệu mẫu lành tính* khai phá bằng PrefixSpan [27] trên cửa sổ huấn luyện bình thường.
- Đặc trưng thời gian (Δt) bị *tắt* vì DongTing (`strace -v -f`) không có dấu thời gian theo từng syscall (`has_temporal=false`).

Chuẩn hóa bằng RobustScaler với phân vị (1,99), *chỉ* khớp trên cửa sổ huấn luyện bình thường để tránh rò rỉ dữ liệu tấn công.



Hình 2: Quy trình kỹ thuật đặc trưng: từ chuỗi syscall thô → lọc theo bảng chữ cái cảm biến → cửa sổ trượt (15, 3) → véc-tơ 43 chiều.

3.5 Mô hình SyscallAD

- **VAE** [25]: hidden_dim=32, latent_dim=8, huấn luyện *chỉ* trên cửa sổ bình thường; điểm bất thường là lỗi tái cấu trúc. Huấn luyện đa hạt giống (seed 0,1,2), giữ hạt giống có AUC kiểm định tốt nhất để xử lý hiện tượng *sụp đổ hậu nghiệm* (posterior collapse).
- **iForest** [26]: 300 cây, contamination=0,02, cũng huấn luyện trên dữ liệu bình thường.
- **Hợp nhất tập hợp**: $s = 0,7 \cdot \text{RobustScale}(s_{\text{VAE}}) + 0,3 \cdot \text{RobustScale}(s_{\text{iForest}})$.

Syscall đặc quyền — tín hiệu hành vi của leo thang đặc quyền		
Các syscall hiếm trong tài bình thường được biểu diễn bằng bit hiện diện (đặc trưng disc) — mỗi nhóm tương ứng một vector tấn công		
Nhóm hành vi leo thang đặc quyền	Syscall đặc quyền (đặc trưng disc)	Ví dụ CVE / kỹ thuật
Nâng định danh & capability	<code>setuid</code> <code>setgid</code> <code>capset</code> <code>prctl</code>	CVE-2021-4034 (PwnKit) — chiếm quyền root cục bộ
Phá vỡ cô lập namespace / thoát container	<code>unshare</code> <code>setns</code> <code>pivot_root</code> <code>mount</code>	CVE-2019-5736 (runc) — ghi đè runtime, thoát container
Ghi đè dữ liệu chỉ-đọc	<code>splice</code>	CVE-2022-0847 (Dirty Pipe) — sửa file đặc quyền
Tiêm / điều khiển tiến trình khác	<code>ptrace</code>	Process injection, đánh cắp phiên đặc quyền
Nạp mã vào nhân	<code>init_module</code> <code>finit_module</code> <code>bpf</code>	Kernel rootkit, lạm dụng eBPF đọc/ghi nhân
Thực thi mã bất thường (fileless)	<code>memfd_create</code> <code>execve</code>	Spawn shell, chạy payload không chạm đĩa

Một bit hiện diện bật khi syscall xuất hiện dù chỉ một lần trong cửa sổ — bên với pha loãng giữa nhiều syscall thông thường.

Hình 3: Ánh xạ các syscall đặc quyền (đặc trưng disc) sang nhóm hành vi leo thang đặc quyền và ví dụ CVE thực. Mỗi nhóm là một vector tấn công vượt ranh giới tenant; bit hiện diện của một syscall đặc quyền là tín hiệu phát hiện cốt lõi của cơ chế.

3.6 Chọn ngưỡng (không rò rỉ)

Ngưỡng phát hiện được chọn *chỉ* trên tập kiểm định và lưu vào `results.json`; tập kiểm tra không tham gia chọn ngưỡng. Khi phục vụ trực tiếp, ngưỡng vận hành được hiệu chỉnh lại theo phân phối điểm quan sát trên cụm (xem Mục 5).

3.7 Mở rộng: bộ phát hiện trực tiếp

Thành phần suy luận đọc luồng sự kiện syscall từ Tetragon [3] qua gRPC, gom theo tiến trình, cắt cửa sổ (15,3) và trích *đúng* véc-tơ 43 chiều như lúc huấn luyện (được kiểm tra ngang hàng bằng `tests/test_feature_parity.py`), rồi chấm điểm bằng các mô hình đã lưu. Cảnh báo và chỉ số được phơi bày qua Prometheus và trực quan hóa trên Grafana; nhật ký sự kiện Tetragon được thu thập qua Loki/Promtail. Toàn bộ chạy bằng Docker/Kubernetes, không cài công cụ trực tiếp lên máy chủ.

4 Triển khai và thiết lập thực nghiệm

4.1 Nguyên tắc: mọi công cụ chạy bằng Docker

Để bảo đảm khả năng tái lập và không làm “bẩn” máy chủ, toàn bộ chuỗi công cụ (huấn luyện, suy luận, biên dịch báo cáo) đều chạy trong container. Quy trình huấn luyện được đóng gói bằng `docker-compose`; bộ phát hiện trực tiếp, Prometheus và Grafana cũng khởi chạy bằng `docker-compose`; cụm Kubernetes thử nghiệm cài Tetragon qua script.

4.2 Huấn luyện ngoại tuyến

Một nguồn chân lý duy nhất là `train.py` (mọi hàm học máy, không vẽ đồ thị); số tay `train_dongting.ipynb` nhập lại từ `train.py` và bổ sung đồ thị + bình luận học thuật. Hai chế độ luôn đồng bộ: muốn đổi thuật toán chỉ sửa `train.py`.

```
cd DeSFAM/training
# ạChạy nhanh, không ươtng tác:
docker compose -f docker-compose.pipeline.yml run --rm train
# ạHoc ươtng tác (Jupyter Lab :8888):
docker compose -f docker-compose.pipeline.yml up jupyter
```

Đầu ra (lưu vào `../model/`) gồm: bộ chuẩn hóa đặc trưng, mô hình iForest, trọng số bộ mã hóa/giải mã VAE, các bộ chuẩn hóa điểm thành phần, tham số tập hợp ($\alpha = 0,7$), từ vựng đặc trưng (`fe_report.json`), và `results.json` (chỉ số + ngưỡng + siêu tham số).

4.3 Cấu hình tái lập (ghi trong `results.json`)

Bảng 1 liệt kê cấu hình thí nghiệm được lưu vết tự động, giúp người đánh giá đối chiếu chính xác.

Bảng 1: Cấu hình thí nghiệm tái lập (trích `results.json`).

Tham số	Giá trị
Thí nghiệm	<code>dongting_15_3</code>
Số chuỗi	18 874
Cửa sổ trượt (độ dài, bước)	(15, 3)
Số cửa sổ huấn luyện / kiểm định / kiểm tra	90 456 / 896 581 / 687 926
Số chiều đặc trưng	43
VAE latent_dim / hidden_dim	8 / 32
iForest contamination	0,02
Đặc trưng thời gian	không (<code>has_temporal=false</code>)
Số mẫu lành tính (PrefixSpan)	150

4.4 Triển khai trực tiếp trên multi-tenant Kubernetes

Cụm thử nghiệm mô phỏng môi trường nhiều người thuê bằng cách cô lập tenant theo namespace (mỗi namespace là một tenant); cảm biến giám sát syscall *theo từng namespace* để quy kết hành vi về đúng tenant. Cảm biến syscall là Tetragon [3], cài qua script và áp một chính sách truy vết (syscall-ad-tracing) chỉ phát ra 23 syscall an ninh (Mục 3). Bộ phát hiện đọc sự kiện qua gRPC NodePort.

```
cd DeSFAM/Kubernetes
bash tetragon/install.sh # cài Tetragon + NodePort
kubectl apply -f tetragon/tracing-policy.yaml # chính sách 23 syscall
bash deploy.sh # workload lành tính + tan cong
```

```
cd DeSFAM/inference
# Detector + Prometheus + Grafana, dùng gương van hành đa hiệu chỉnh:
docker compose --env-file .env.uat up -d --build
```

4.5 Khối lượng công việc và kịch bản tấn công

Khối lượng làm việc tiêu biểu là MongoDB (truy vấn CRUD bình thường). Kịch bản tấn công trong không gian tên demo mô phỏng các chuỗi đặc quyền (thực thi shell bất thường, ptrace, nạp mô-đun, thao tác mount/unshare) phản ánh các họ CVE thoát container và leo thang đặc quyền như CVE-2021-4034 [30], CVE-2022-0847 [31], CVE-2019-5736 [32]. Mục tiêu là quan sát *độ phân tách điểm* giữa hành động và tấn công khi phục vụ trực tiếp, chứ không tuyên bố chặn 14/14 CVE như công trình tham chiếu (xem Mục 5.4).

4.6 Quan sát

Grafana (:3000) hiển thị điểm bất thường theo thời gian và nhật ký sự kiện Tetragon (qua Loki); Prometheus (:9090) lưu chuỗi thời gian của điểm số và cảnh báo. Đây là kênh quan sát bằng chứng cho phần triển khai trực tiếp (Mục 5).

5 Kết quả và đánh giá

5.1 Hiệu lực phát hiện ngoại tuyến trên DongTing

5.1.1 Hiệu năng từng mô hình và tập hợp

Bảng 2 trình bày kết quả trên tập kiểm tra DongTing (lấy trực tiếp từ `results.json`). VAE đạt AUC kiểm định cao và ổn định qua ba hạt giống (0,9426 / 0,9438 / 0,9430; chọn hạt giống 1). Trên tập kiểm tra, tập hợp đạt AUC 0,835 và Average Precision 0,990.

Bảng 2: Hiệu năng SyscallAD trên tập kiểm tra DongTing (cửa sổ 15/3, 43 chiều). Ngưỡng chọn trên tập kiểm định.

Mô hình	AUC	AP	F1	Precision	Recall
iForest	0,851	0,991	0,474	0,999	0,311
VAE	0,734	0,985	0,421	0,999	0,267
Tập hợp	0,835	0,990	0,417	0,999	0,264
VAE (AUC kiểm định, hạt giống tốt nhất)	0,944	—	—	—	—

5.1.2 Đối chiếu với công trình tham chiếu

Bảng 3 đối chiếu kết quả của cơ chế đề xuất với mức công bố của công trình tham chiếu DeSFAM [1]. Có *tương đồng* ở hai điểm: (i) AUC kiểm định của VAE thậm chí cao hơn

mức công bố cho thành phần VAE; (ii) độ chính xác (precision) rất cao ($\approx 0,999$), khớp định hướng “ít báo động giả”. Tuy nhiên *còn khác biệt* ở F1/Recall so với mức công bố: F1 tập hợp của chúng tôi 0,417 so với 0,92 trong công trình tham chiếu, do recall thấp (0,264) tại ngưỡng bảo thủ chọn trên tập kiểm định (ngưỡng ưu tiên precision). Nói cách khác, mô hình *xếp hạng* bất thường tốt (AUC/AP cao) nhưng *điểm chốt nhị phân* tại một ngưỡng cố định lại bỏ sót nhiều cửa sổ tấn công.

Bảng 3: Cơ chế đề xuất so với số liệu công bố của công trình tham chiếu DeSFAM (lỗi phát hiện bất thường).

Chỉ số	DeSFAM (công bố)	Cơ chế đề xuất	Nhận định
AUC (tập hợp)	0,94	0,835	Thấp hơn nhưng cùng bậc
AP	0,87	0,990	Cao hơn (dữ liệu mất cân bằng)
F1 (tấn công)	0,92	0,417	Còn khác biệt —do ngưỡng ưu tiên precision
VAE (AUC kiểm định)	0,747*	0,944	Cao hơn mức công bố cho VAE
Precision	cao	0,999	Tái lập định hướng ít báo động giả

*Giá trị thành phần VAE theo Bảng 1 của công trình tham chiếu; tập hợp công bố $\approx 0,94$.

5.2 Triển khai trực tiếp trên multi-tenant Kubernetes

Khi phục vụ trực tiếp, mô hình huấn luyện ngoại tuyến giữ được *khả năng phân tách* giữa lạnh tính và tấn công—kết quả quan trọng nhất của chuyên đề:

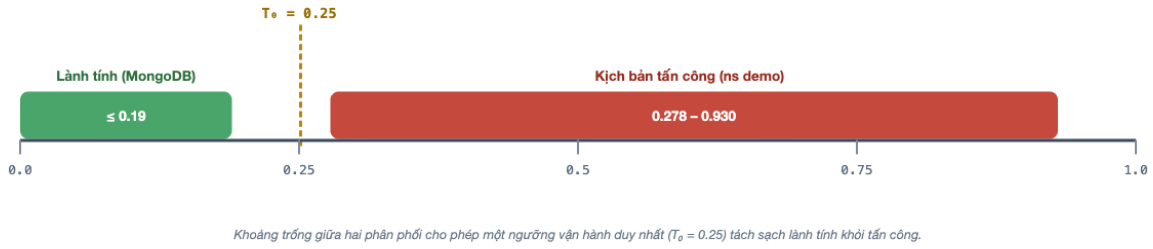
- Khối lượng lạnh tính (MongoDB) có điểm bất thường $\leq 0,19$.
- Các kịch bản tấn công trong không gian tên demo có điểm 0,278–0,930.

Khoảng trống giữa hai phân phối cho phép đặt một **ngưỡng vận hành** $T_0 = 0,25$ (lưu trong `inference/.env.uat`) tách sạch hai lớp trong môi trường thử nghiệm (Hình 4). Đây là bằng chứng cho thấy thiết kế DeSFAM khả thi ngoài mô phỏng: cùng một không gian đặc trưng 43 chiều, sinh ngoại tuyến từ DongTing và trực tuyến từ Tetragon, tạo ra điểm số tương thích.

Bảng 4: Phân tách điểm bất thường khi phục vụ trực tiếp (Tetragon/K8s).

Nguồn	Khoảng điểm	So với $T_0 = 0,25$
MongoDB (lạnh tính)	$\leq 0,19$	Dưới ngưỡng (không báo động)
Kịch bản tấn công (demo)	0,278–0,930	Trên ngưỡng (báo động)

Phân tách điểm bất thường khi phục vụ trực tiếp (Tetragon / Kubernetes)



Hình 4: Phân tách điểm bất thường khi phục vụ trực tiếp qua Tetragon trên Kubernetes. Khối lượng lành tính (MongoDB) nằm dưới ngưỡng vận hành $T_0 = 0,25$, các kịch bản tấn công nằm trên—khoảng trống đủ rộng để một ngưỡng duy nhất tách sạch hai lớp.

5.3 Điều kiện thành công của cơ chế

Yếu tố quyết định là **căn chỉnh bảng chữ cái cảm biến** giữa huấn luyện và phục vụ. Khi huấn luyện trên toàn bộ vũ trụ syscall (gồm read/write/close/futex mật độ cao) trong khi Tetragon chỉ phát 23 syscall an ninh, mọi cửa sổ trực tiếp trở nên thừa và *suy biến*: điểm lành tính $\approx$ điểm tấn công, không thể đặt ngưỡng. Sau khi buộc `train.py` lọc DongTing về đúng 23 syscall trước khi cắt cửa sổ (`SENSOR_ALPHABET=1`), AUC kiểm định của VAE tăng lên 0,944 (giải quyết sụp đổ hậu nghiệm) và đạt được độ phân tách trực tiếp ở Bảng 4. Bài học: trong các hệ phát hiện bất thường *huấn luyện-rồi-phục-vụ*, sự đồng nhất không gian đặc trưng quan trọng ngang—thậm chí hơn—kiến trúc mô hình. Tính đồng nhất này được ràng buộc bằng kiểm thử ngang hàng `test_feature_parity.py` (đạt) để `train.py` và bộ trích đặc trưng khi phục vụ sinh véc-tơ *trùng khít từng bit*.

5.4 Thảo luận và đánh giá

5.4.1 Cơ chế đạt được gì so với công trình tham chiếu

Cơ chế đề xuất tái hiện được *định hướng và tính khả thi* của lõi phát hiện tham chiếu DeSFAM: bộ phát hiện VAE + iForest xếp hạng bất thường tốt (AUC kiểm định VAE 0,944; AP tập hợp 0,990) và giữ khả năng phân tách khi phục vụ trực tiếp qua Tetragon. Điều *chưa* đạt là các con số tuyệt đối ở chế độ phân loại nhị phân—đặc biệt F1 0,92 và recall cao của công trình tham chiếu. Nguyên nhân khả dĩ: (i) công trình tham chiếu chọn ngưỡng tối ưu F1 hoặc đánh giá trên một phân chia tấn công/khối lượng khác; (ii) khác biệt về phiên bản DongTing, cách cắt cửa sổ và tập syscall đầu vào; (iii) công trình tham chiếu bổ sung Giai đoạn 3 (thực thi trong nhân) để đạt “chặn 14/14 CVE”—một chỉ số *thực thi*, không phải chỉ số phân loại của riêng SyscallAD.

5.4.2 Ý nghĩa của khoảng cách ngưỡng

Việc AUC/AP cao nhưng F1 thấp tại ngưỡng cố định cho thấy giá trị của *hiệu chỉnh ngưỡng theo môi trường*. Ngưỡng bảo thủ chọn trên tập kiểm định (0,81 cho điểm tập hợp đã chuẩn hóa) tối đa hóa precision nên bỏ sót cửa sổ tấn công “nhẹ”. Ngược lại,

khi phục vụ trực tiếp, ngưỡng vận hành $T_0 = 0,25$ được hiệu chỉnh theo phân phối quan sát giúp tách sạch hai lớp. Đây là một phát hiện thực dụng: với phát hiện bất thường, hiệu chỉnh ngưỡng tại chỗ quan trọng ngang với chất lượng mô hình.

5.4.3 Lựa chọn thiết kế khác với công trình tham chiếu

Chúng tôi dùng Tetragon [3] làm cảm biến eBPF thay vì tự viết chương trình eBPF/LSM. Điều này đẩy nhanh hiện thực hóa và bám hệ sinh thái Kubernetes thực tế, nhưng cũng *thu hẹp* bằng chữ cái syscall về 23 lời gọi an ninh—một ràng buộc kéo theo phải lọc dữ liệu huấn luyện tương ứng (Mục 5). Việc thực thi/chặn trong nhân (Giai đoạn 3 đầy đủ của DeSFAM) chưa được hiện thực hóa; chuyên đề dừng ở phát hiện và cảnh báo.

5.4.4 Hạn chế và đe dọa tính hợp lệ

- **Hợp lệ nội tại:** kịch bản tấn công trực tiếp là *mô phỏng* chuỗi syscall đặc quyền, không phải khai thác CVE thực; do đó độ phân tách ở Bảng 4 là dấu hiệu khả quan chứ chưa phải bằng chứng chặn khai thác thật.
- **Hợp lệ ngoại tại:** một khối lượng lành tính (MongoDB) và một không gian tên thử nghiệm chưa đại diện cho đa dạng khối lượng sản xuất; cần mở rộng (Nginx, Redis, Node.js...) như công trình tham chiếu.
- **Hợp lệ kết luận:** mất cân bằng lớp lớn (tỉ lệ tấn công cao trong tập kiểm tra của số) khiến AP cao dễ gây hiểu nhầm; chúng tôi báo cáo kèm AUC, precision, recall để tránh kết luận một chiều.
- **Hợp lệ cấu trúc:** đặc trưng thời gian bị tắt do DongTing không có dấu thời gian theo syscall; mô hình có thể thiếu một tín hiệu mà công trình tham chiếu tận dụng.

5.4.5 Hướng cải thiện

(1) Chọn ngưỡng tối ưu F1 và báo cáo đường cong precision–recall đầy đủ; (2) hiện thực hóa Giai đoạn 3 (chặn bằng eBPF/LSM hoặc Tetragon Enforcement) để đo “tỉ lệ chặn” so sánh trực tiếp với công trình tham chiếu; (3) mở rộng tập khối lượng và dùng khai thác CVE thực trong môi trường cách ly; (4) bổ sung đặc trưng thời gian khi dùng cảm biến có dấu thời gian (Tetragon cung cấp), điều DongTing thiếu.

6 Kết luận

Chuyên đề đã nghiên cứu và hiện thực hóa một *cơ chế phát hiện tấn công leo thang đặc quyền* cho môi trường multi-tenant Kubernetes dựa trên phân tích lời gọi hệ thống. Cơ chế kế thừa thiết kế lõi phát hiện bất thường SyscallAD (VAE + Rừng cô lập) của công trình tham chiếu DeSFAM [1], định vị các đặc trưng syscall đặc quyền làm tín hiệu cho leo thang đặc quyền; huấn luyện trên bộ dữ liệu công khai DongTing và được triển khai thành một bộ phát hiện chạy trực tiếp bằng Tetragon/eBPF giám sát theo namespace trên Kubernetes, toàn bộ đóng gói bằng Docker để bảo đảm tái lập.

Các kết quả chính như sau. Về *phát hiện ngoại tuyến*, lỗi phát hiện đạt định hướng và khả năng xếp hạng tốt (VAE AUC kiểm định 0,944, AP tập hợp 0,990, precision 0,999) nhưng còn khác biệt về F1/recall so với mức công bố do ngưỡng ưu tiên precision. Về *triển khai trực tiếp*, mô hình giữ được khả năng phân tách khi phục vụ trực tiếp (lành tính $\leq 0,19$ so với các kịch bản leo thang đặc quyền 0,278–0,930), cho phép ngưỡng vận hành $T_0 = 0,25$. Về *điều kiện thành công*, yếu tố then chốt là căn chỉnh không gian đặc trưng (bảng chữ cái cảm biến 23 syscall) giữa huấn luyện và phục vụ, được ràng buộc bằng kiểm thử ngang hàng.

Tóm lại, báo cáo trình bày một cơ chế phát hiện khả thi cho leo thang đặc quyền trong multi-tenant Kubernetes, kèm một đối chiếu *trung thực* với công trình tham chiếu: chỉ rõ điều gì tương đồng, điều gì khác biệt và vì sao, đồng thời lấp khoảng cách “mô phỏng $\rightarrow$ triển khai” bằng một bộ phát hiện trực tiếp. Hướng tiếp theo gồm chọn ngưỡng tối ưu F1, hiện thực hóa khâu chặn trong nhân (Giai đoạn 3) để *ngăn chặn* chứ không chỉ phát hiện, mở rộng khối lượng công việc đa tenant và dùng khai thác CVE thực trong môi trường cách ly.

Tài liệu tham khảo

- [1] S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, “DeSFAM: An adaptive eBPF and AI-driven framework for securing cloud containers in real time,” *IEEE Access*, vol. 13, 2025.
- [2] G. Duan, Y. Fu, M. Cai, H. Chen, and J. Sun, “DongTing: A large-scale dataset for anomaly detection of the Linux kernel,” *Journal of Systems and Software*, vol. 203, p. 111768, Sep. 2023.
- [3] Cilium / Isovalent, “Tetragon: eBPF-based security observability and runtime enforcement,” <https://tetragon.io/>, 2024.
- [4] O. Jarkas, R. Ko, N. Dong, and R. Mahmud, “A container security survey: Exploits, attacks, and defenses,” *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025.
- [5] S. Sultan, I. Ahmad, and T. Dimitriou, “Container security: Issues, challenges, and the road ahead,” in *IEEE Access*, vol. 7, 2019, pp. 52 976–52 996.
- [6] T. Combe, A. Martin, and R. Di Pietro, “To docker or not to docker: A security perspective,” *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016.
- [7] R. Shu, X. Gu, and W. Enck, “A study of security vulnerabilities on docker hub,” in *Proceedings of the 7th ACM on Conference on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280.
- [8] M. Rostamipoor, S. Ghavamnia, and M. Polychronakis, “Confine: Fine-grained system call filtering for container attack surface reduction,” *Computers & Security*, vol. 132, p. 103338, Sep. 2023.
- [9] S. Kim, B. J. Kim, and D. H. Lee, “Prof-gen: Practical study on system call whitelist generation for container attack surface reduction,” in *Proceedings of the*

- IEEE 14th International Conference on Cloud Computing (CLOUD)*, Sep. 2021, pp. 278–287.
- [10] S. Yang, B. B. Kang, and J. Nam, “Optimus: Association-based dynamic system call filtering for container attack surface reduction,” *Journal of Cloud Computing*, vol. 13, no. 1, p. 71, Mar. 2024.
 - [11] J. Her, C. H. Jo, T. Park, and S. Lee, “KubeRosy: A dynamic system call filtering framework for containers,” *IEEE Access*, vol. 12, pp. 159 889–159 901, 2024.
 - [12] S. Song, S. Suneja, M. V. Le, and B. Tak, “On the value of sequence-based system call filtering for container security,” in *Proceedings of the IEEE 16th International Conference on Cloud Computing (CLOUD)*, Jul. 2023, pp. 296–307.
 - [13] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, “A sense of self for Unix processes,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 1996, pp. 120–128.
 - [14] S. A. Hofmeyr, S. Forrest, and A. Somayaji, “Intrusion detection using sequences of system calls,” *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.
 - [15] C. Warrender, S. Forrest, and B. Pearlmutter, “Detecting intrusions using system calls: Alternative data models,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 1999, pp. 133–145.
 - [16] D. Wagner and P. Soto, “Mimicry attacks on host-based intrusion detection systems,” in *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, 2002, pp. 255–264.
 - [17] G. Creech and J. Hu, “A semantic approach to host-based intrusion detection systems using contiguous and discontiguous system call patterns,” *IEEE Transactions on Computers*, vol. 63, no. 4, pp. 807–819, 2014.
 - [18] A. S. Abed, C. Clancy, and D. S. Levy, “Applying bag of system calls for anomalous behavior detection of applications in Linux containers,” in *Proceedings of the IEEE Globecom Workshops*, 2015.
 - [19] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, “Deep autoencoding Gaussian mixture model for unsupervised anomaly detection,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
 - [20] L. Ruff, R. A. Vandermeulen, N. Görnitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, “Deep one-class classification,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 4393–4402.
 - [21] W. Haider, J. Ahmad, M. Nasim, Z. A. Baig, and A. A. Laghari, “Methods for host-based intrusion detection with deep learning,” *Digital Threats: Research and Practice*, vol. 2, no. 4, pp. 1–29, 2021.
 - [22] P. Castanhel, P. Pinto, and F. Magalhães, “Taking a peek: An evaluation of anomaly detection using system calls for containers,” in *Proceedings of the IEEE International Symposium on Computers and Communications (ISCC)*, 2021.

- [23] M. El Khairi, D. Colle, F. De Turck, and B. Volckaert, “Contextualizing system calls in containers for anomaly-based intrusion detection,” in *Proceedings of the ACM Cloud Computing Security Workshop (CCSW)*, 2022.
- [24] Q. Bsoul, M. Al-Quraan, and F. Al-Turjman, “Ensemble of random and isolation forests for graph-based intrusion detection in containers,” in *Proceedings of the IEEE International Conference on Cyber Security and Resilience (CSR)*, 2022.
- [25] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” in *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*, 2014.
- [26] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [27] J. Pei, J. Han, B. Mortazavi-Asl, J. Wang, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, “Mining sequential patterns by pattern-growth: The PrefixSpan approach,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 11, pp. 1424–1440, 2004.
- [28] B. Gregg, “BPF performance tools: Linux system and application observability,” *Addison-Wesley Professional*, 2019, iISBN: 978-0-13-655482-1.
- [29] Cilium Authors, “Cilium: eBPF-based networking, observability, and security,” <https://cilium.io/>, 2024.
- [30] NIST National Vulnerability Database, “CVE-2021-4034: Polkit local privilege escalation,” <https://nvd.nist.gov/vuln/detail/CVE-2021-4034>, 2022.
- [31] —, “CVE-2022-0847: Dirty pipe — linux kernel privilege escalation,” <https://nvd.nist.gov/vuln/detail/CVE-2022-0847>, 2022.
- [32] —, “CVE-2019-5736: Docker runc container escape,” <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>, 2019.

A Phụ lục Tái lập Kết quả

Phụ lục này cho phép một người đánh giá độc lập tái lập kết quả. Mọi công cụ chạy bằng Docker—không cần cài đặt trực tiếp lên máy chủ (ngoài Docker và, cho phần trực tiếp, một cụm Kubernetes có quyền nạp eBPF).

A.1 Môi trường

```
# Yêu cầu: Docker; (phần trực tiếp) một cụm Kubernetes + Tetragon.
git clone <REPO_URL> && cd kma-chuyen-de
# Dữ liệu DongTing: nạp từ các zip trong DongTing/DB (xem README).
```

A.2 Huấn luyện ngoại tuyến (tái lập Bảng 2)

```
cd DeSFAM/training
docker compose -f docker-compose.pipeline.yml run --rm train
# Ket qua so + nguong duoc ghi vao ../model/results.json
```

A.3 Triển khai trực tiếp (tái lập Bảng 4)

```
cd DeSFAM/Kubernetes
bash tetragon/install.sh
kubectl apply -f tetragon/tracing-policy.yaml # 23 syscall an ninh
bash deploy.sh                               # workload lanh tinh + tan cong

cd ../inference
docker compose --env-file .env.uat up -d --build # detector + Prometheus + Grafana
# Grafana: http://localhost:3000 (admin/admin)
```

A.4 Kiểm tra tính nhất quán đặc trưng

```
cd DeSFAM
# Bao dam train.py va bo trich dac trung khi phuc vu sinh vec-to trung khi:
docker compose -f training/docker-compose.pipeline.yml run --rm \
  train python -m pytest tests/test_feature_parity.py -q
```

A.5 Biên dịch báo cáo này (Docker)

```
cd Report
./build.sh # chay XeLaTeX qua Docker texlive -> main.pdf
```